

WIPRO CAPSTONE PROJECT

Virtual **Ultrasonic** Parking Sensor

A Linux device-driver based simulation of a car parking sensor, with a C++17 monitoring application and TCP status client.

C++17

C Kernel Driver

Linux

Device Drivers

TCP Sockets

What is this project?

Problem

A real ultrasonic parking sensor measures distance to obstacles. We simulate it entirely in software.

Driver layer

A Linux kernel module generates distance values (150 cm → 10 cm → back) via a timer.

Application layer

A C++17 app reads the driver, classifies SAFE / WARNING / DANGER, and logs it.

Client layer

A TCP client displays the live status: STATUS distance=37 state=WARNING.

System Architecture



Separation of concerns

The driver only reports numbers; deciding WARNING vs DANGER is user-space policy.

Simulate mode

--simulate swaps the driver for an in-process sensor — full demo without root.

Distance → State Mapping

`distance > 50 cm`

SAFE

`21 cm ≤ distance ≤ 50 cm`

WARNING

`distance ≤ 20 cm`

DANGER

Project Structure

driver/

`parking_sensor.c` — kernel module (insmod/rmmod, /dev node, timer)

app/

monitor: config, sensor reader, state manager, logger, TCP server

client/

`status_client.cpp` — reads live status over TCP

tests/

41 tests / 1140 checks, custom harness, no framework needed

Key Design Decisions

Interface-driven

ISensorSource + ThresholdPolicy make hardware and policy swappable/testable.

No dependencies

Only g++, make and POSIX — builds anywhere Linux runs.

Failures visible

Missing device, bad config or failed read prints a message and a counter, never a silent wrong state.

Clean shutdown

SIGINT/SIGTERM, --iterations, or daemon mode all stop safely and flush the log.

Build & Run

```
$ make                # build driver, app, client, tests
$ make test           1140 checks run, 0 failed
$ ./build/parking_monitor --simulate --iterations 20
    [monitor] STATE CHANGE -> SAFE at 140 cm
$ ./build/status_client --port 9000 --count 3
    STATUS distance=37 state=WARNING
```

Testing & Verification

Unit tests

Threshold boundaries, config parsing, logger formatting, telemetry records.

Integration test

A full monitor run must write SAFE, WARNING and DANGER lines to the log.

TCP smoke test

test_all.sh starts the server and checks the client receives live status.

Result

1140 checks, 0 failures — the suite runs with `make test`.

Conclusion

What we learned

Linux device drivers, character devices, threads, TCP sockets, layered design.

Outcome

A working, tested, documented simulation with a virtual sensor

Repository

github.com/rajarp3108-tech/Virtual_Sonic_Parking

Future scope

Real ultrasonic hardware (HC-SR04) via GPIO, dashboard UI, alerts